Object Oriented Programming

CS22301

UNIT III

EXCEPTION HANDLING and I/O

in

JAVA



A. Bamila Virgin Louis

Asso.Prof/ CSE

SXCCE

UNIT III

Course Objective

- To define exceptions and use I/O streams

Course Outcome

CO3: Develop Java applications using exception handling techniques and I/O operations.

UNIT III

Exceptions - exception hierarchy - throwing and catching exceptions – built-in exceptions, creating own exceptions, Stack Trace Elements. Input / Output Basics – Streams – Byte streams and Character streams – Reading and Writing Console – Reading and Writing Files.

INTRODUCTION:

- An Error is a severe situation generated when the user performs an unexpected operation
- An Exception is an event that occurs during the program execution and disrupts the normal flow of the program's execution.
- Exception is an abnormal condition in a program or it's a run-time error.
- Errors mostly happen at compile-time like syntax error; however it can happen at runtime as well. Whereas an Exception occurs at runtime (checked exceptions can be detected at compile time). Both Error and Exception have the same parent class as Throwable.
- Errors must be checked and handled manually in computer languages that do not support exception handling.

Exception-Handling Fundamentals

- Java's exception handling mechanisms support run-time error management.
- A Java exception is an **object** that describes an exceptional condition that has occurred in a piece of code.
- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- That method may choose to handle the exception itself, or pass it on.
- Either way, at some point, the exception is caught and processed.

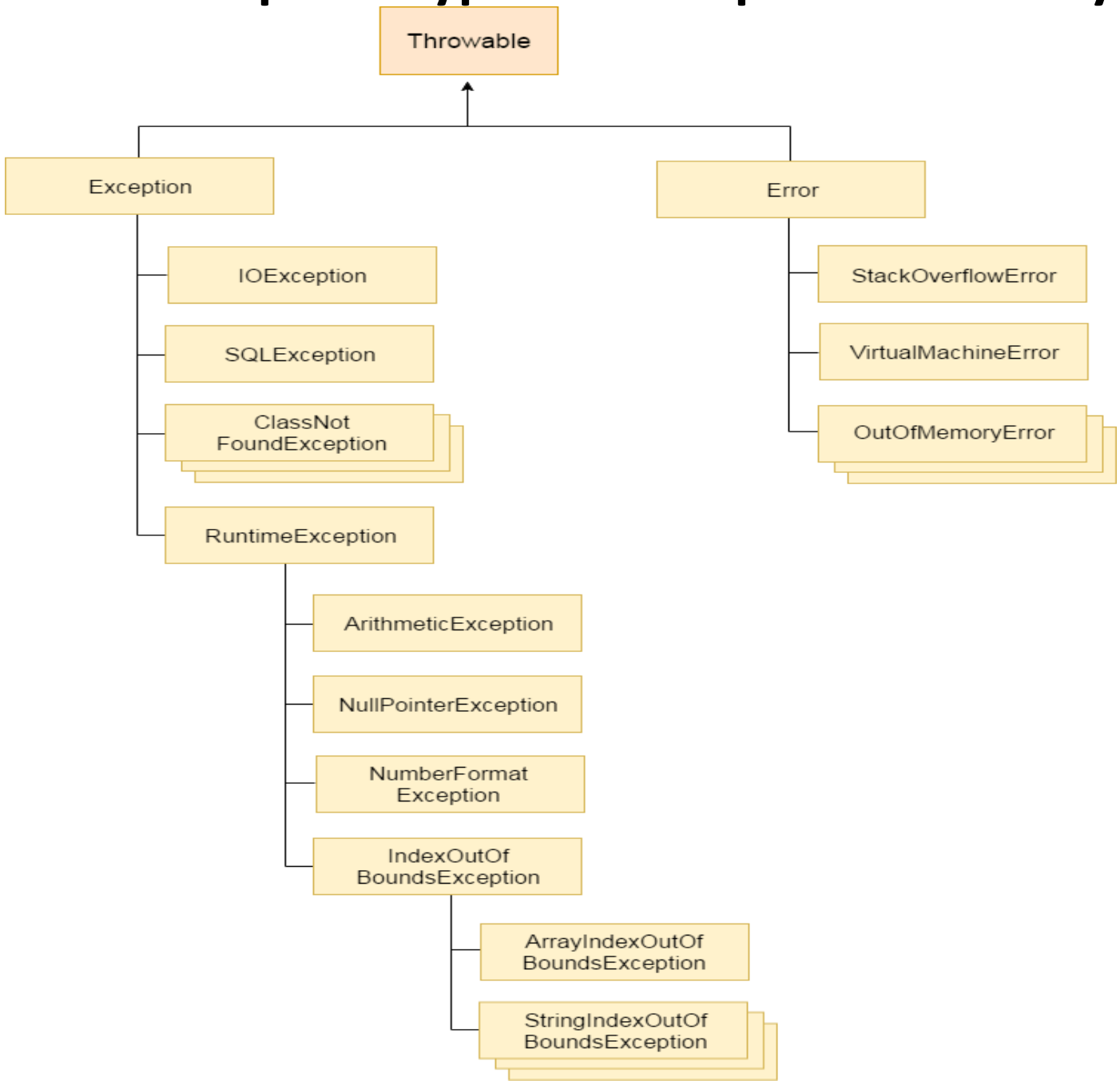
Java exception handling is managed via five keywords: **try, catch, throw, throws, and finally**.

- **try** - monitor for exceptions
- **catch** - catch this exception using catch and handle it.
- **throw** - System-generated exceptions are automatically thrown by the Java run-time system. To manually throw an exception, use the keyword throw.
- **throws** - Any exception that is thrown out of a method must be specified by a throws clause.
- **finally** – Keep the code that absolutely must be executed after a try block completes

General form of an exception-handling block:

```
try
{
        // block of code to monitor for errors
}
catch (ExceptionType1 exOb)
{
        // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb)
 {
        // exception handler for ExceptionType2
}
// ...
finally
{
        // block of code to be executed after try block ends
}
```

Exception Types or Exception Hierarchy



Java's Built-in Exceptions

- Inside the standard package java.lang, Java defines several exception classes.

Types of Exceptions:

- **Checked exceptions** – A checked exception is an exception that occurs at the compile time, these are also called as compile time exceptions. These exceptions cannot simply be ignored at the time of compilation, the programmer should take care of (handle) these exceptions.

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

Unchecked exceptions −

- An unchecked exception is an exception that occurs at the time of execution.
- These are also called as **Runtime Exceptions**.
- These include programming bugs, such as logic errors or improper use of an API.
- Runtime exceptions are ignored at the time of compilation.

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBounds	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

TABLE 10-1 Java's Unchecked **RuntimeException** Subclasses Defined in **java.lang**

Uncaught Exceptions

Consider the small program which includes an expression that intentionally causes a divide-by-zero error:

```
class Exc
 {
        public static void main(String args[])
        {
                int d = 0;
                int a = 42 / d;
        }
}
```

- Java run-time system detects the attempt to divide by zero
- it constructs a new exception object and then throws this exception.
- the exception is caught by the **default handler** provided by the Java run-time system.

Default Handler:

- Any exception that is not caught by your program will ultimately be processed by the default handler.
- The default handler displays a string describing the exception, prints a stack trace from the point at which the exception occurred, and terminates the program.

Eg. java.lang.ArithmeticException: / by zero

at Exc.main(Exc.java:4)

- the class name, Exc;
- the method name, main;
- the filename, Exc.java;
- the line number, 4,

are all included in the simple stack trace.

- the type of exception thrown is a subclass of Exception called ArithmeticException, which more specifically describes what type of error happened.

- the stack trace will always show the sequence of method invocations that led up to the error. For example, consider the program that introduces the same error but in a method separate from main( ):

```
class Number
{
        void divide()
        {               int a=5/0;
        }
}
class Exc
{
        public static void main(String[] args)
        {               Number n=new Number();
                n.divide();
        }
}
```

Here is the exception generated when this example is executed:

```
java.lang.ArithmeticException: / by zero
at Exc.subroutine(Exc.java:5)
at Exc.main(Exc.java:6)
```

The call stack is quite useful for debugging

Using try and catch

To handle an exception use try and catch clause.

Benefits:

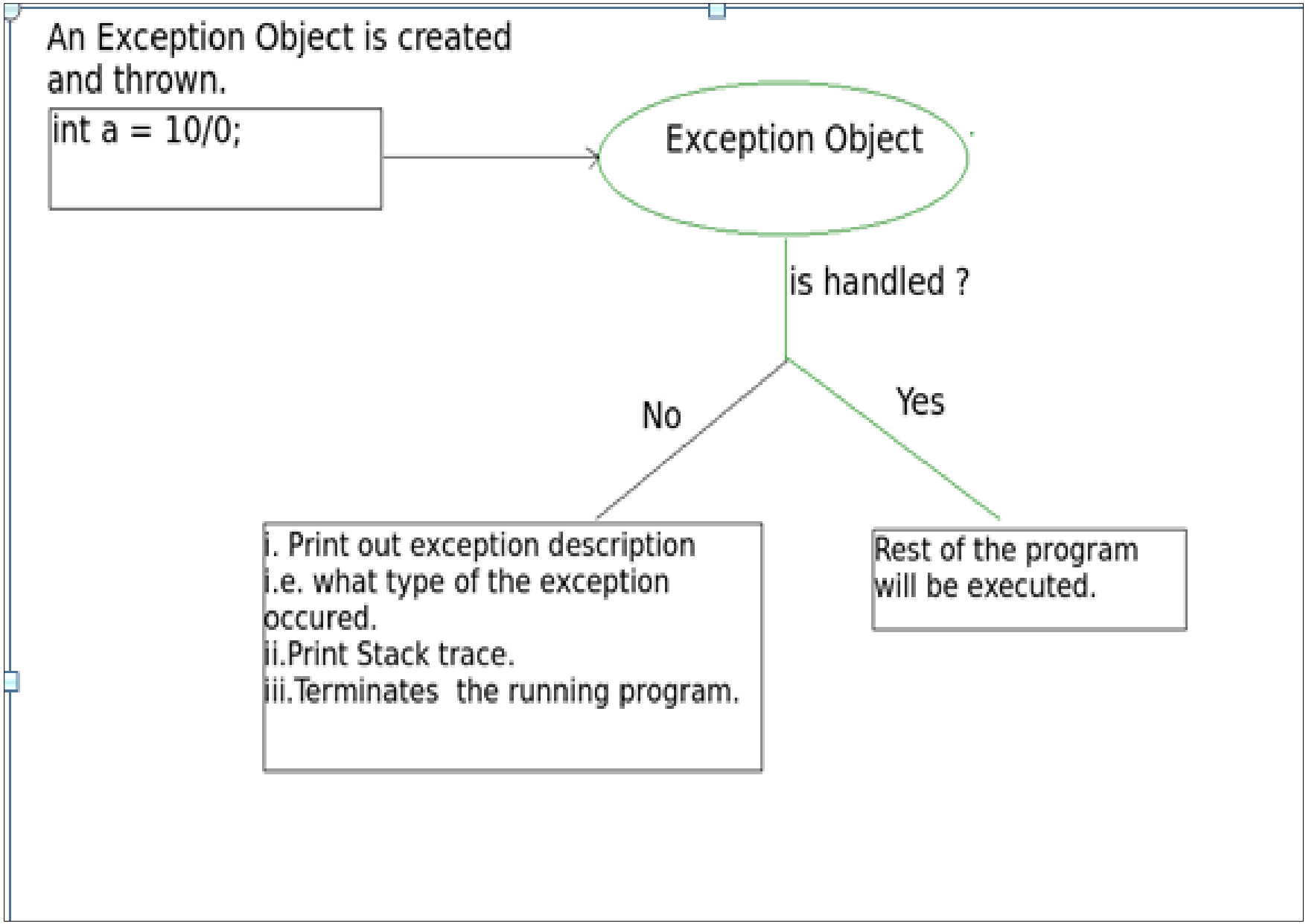
- it allows you to fix the error.
- it prevents the program from automatically terminating.

➢ To handle a run-time error, simply enclose the code that you want to monitor inside a try block.

➢ Immediately following the try block, include a catch clause that specifies the exception type that you wish to catch.

```
class Exc
{
        public static void main(String args[])
        {
                int d, a;
                try
                { // monitor a block of code.
                        d = 0;
                        a = 42 / d;
                        System.out.println("This will not be printed.");
                }
                catch (ArithmeticException e)
                {
                         // catch divide-by-zero error
                        System.out.println("Division by zero.");
                }
                System.out.println("After catch statement.");
        }
}
```

- Once an exception is thrown, program control transfers out of the try block into the catch block and execution never “returns” to the try block from a catch.
- Once the catch statement has executed, program control continues with the next line in the program following the entire try/catch mechanism.
- A try and its catch statement form a unit.
- A catch statement cannot catch an exception thrown by another try statement.
- The statements that are protected by try must be surrounded by curly braces.
- Once the exceptional condition is resolved ,then the program continue on as if the error had never happened.


An Exception Object is created and thrown.
int a = 10/0;
Exception Object
is handled ?
No
Yes
i. Print out exception description i.e. what type of the exception occured.
ii.Print Stack trace.
iii.Terminates the running program.
Rest of the program will be executed.

```
class Exc
{
        public static void main(String args[])
        {
                int d, a,b;
                try
                { // monitor a block of code.
                        d = 0;
                        a = 42 / d;
                        System.out.println("This will not be printed.");
                }
                 catch (ArithmeticException e)
                {
                         // catch divide-by-zero error
                        System.out.println("Division by zero.");
                }
                b=25/5;
                System.out.println(“b=”+b);
        }
}
```

Output:

Division by zero.

b=5

Displaying a Description of an Exception:

Throwable overrides the toString( ) method. so that it returns a string containing a description of the exception.

```
class Exc
{       public static void main(String args[])
        {       int d, a;
                try
                {       d = 0;
                        a = 42 / d;
                         System.out.println("This will not be printed.");
                }
                 catch (ArithmeticException e)
                {       System.out.println(“Exception:"+e);
                }
}}
```

Output:

```
Exception: java.lang.ArithmeticException: / by zero
```

Multiple Catch Blocks

A try block can be followed by multiple catch blocks. The syntax for multiple catch blocks looks like the following −

```
try
{          //statements
}
catch (ExceptionType1 e1)
{          // Catch block
}
catch (ExceptionType2 e2)
{          // Catch block
}
catch (ExceptionType3 e3)
{           // Catch block
}
```

```
public class Test
{
public static void main(String[] args) {
        int ch,a[]={1,2,3};
        Scanner s=new Scanner(System.in);
        S.o.p("Enter choice");
        ch=s.nextInt();
        try
        {
          switch(ch)
          {
             case 1:
                int b=25/0;
                break;
             case 2:
                S.o.p("Array Element"+a[5]);
                break;
          }
         }
        catch(ArithmeticException e)
        {
           S.o.p("Exception "+e);
        }

catch(ArrayIndexOutOfBoundsException e)
        {
              S.o.p("Exception "+e);
        }
    }
}
```

Output:

```
Enter choice
2
Exception java.lang.ArrayIndexOutOfBoundsException: 5
Enter choice
1
Exception java.lang.ArithmeticException: / by zero
```

- When you use multiple catch statements, it is important to remember that exception subclasses must come before any of their super classes.
- This is because a catch statement that uses a superclass will catch exceptions of that type plus any of its subclasses.
- Thus, a subclass would never be reached if it came after its superclass. Further, in Java, unreachable code is an error.

```
class SuperSubCatch
 {
public static void main(String args[])
{
        try
        {       int a = 0;
                int b = 42 / a;
        }
        catch(Exception e)
        {       System.out.println("Generic Exception catch.");
        }
// This catch is never reached because ArithmeticException is a subclass of Exception.
        catch(ArithmeticException e)
         {
                 // ERROR - unreachable
                System.out.println("This is never reached.");
        }
}
}
```

Catching Multiple Type of Exceptions

- Since Java 7, you can handle more than one exception using a single catch block, this feature simplifies the code.

Example:

```
        catch(ArithmeticException | ArrayIndex
OutOfBoundsException e)
{
        //statements
}
```

Nested try Statements

- a try statement can be inside the block of another try.
- Each time a try statement is entered, the context of that exception is pushed on the stack.
- If an inner try statement does not have a catch handler for a particular exception, next try statement's catch handlers are inspected for a match.
- This continues until one of the catch statements succeeds, or until all of the nested try statements are exhausted.
- If no catch statement matches, then the Java run-time system will handle the exception.

General Form:

```
....
try
{
        statement 1;
         statement 2;
         try
        {
          statement 1;
          statement 2;
        }
         catch(Exception e)
        {
         }
}
catch(Exception e)
{
}
....
```

```
public class Number {
   public static void main(String[] args) {
      try{
         try{
            S.o.p("going to divide");
            int b =39/0;
           }
         catch(ArithmeticException e)
         {
           System.out.println(e);
         }

         try{
            int a[]=new int[5];
            a[5]=4;
           }

catch(ArrayIndexOutOfBoundsException e)
         {
           System.out.println(e);
         }
            S.o.p("other statement");
      }
      catch(Exception e)
      {
         System.out.println("handeled");
      }
   System.out.println("normal flow..");
    }
   }
```

output:

```
going to divide
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: 5
other statement
normal flow..
```

- **Nesting of try statements can also occur when method calls are involved**.

```
public class Number {
    public static void main(String[] args) {
        Number n1=new Number();
        try{
            n1.ExcMethod1();
            S.o.p("other stmt");
        }
        catch(Exception e)
        {
            S.o.p("handled");
        }
        S.o.p("normal flow..");
    }
    void ExcMethod1()
    {
        try{
            int a[]=new int[5];
            a[5]=4;
        }

catch(ArrayIndexOutOfBoundsException e)
    {
        System.out.println(e);
    }  }
```

output:

```
java.lang.ArrayIndexOutOfBoundsException: 5
other stmt
normal flow..
```

- **Nesting of try statements can also occur when method calls are involved**.

```
public class Number {
    public static void main(String[] args) {
        Number n1=new Number();
        try{
            n1.ExcMethod1();
            S.o.p("other stmt");
        }
        catch(Exception e)
        {
            S.o.p("handled");
        }
        S.o.p("normal flow..");
    }
    void ExcMethod1()
    {
        try{
            int a[]=new int[5];
            a[5]=4;
        }

catch(ArithmeticException e)
    {
        System.out.println(e);
    } }
```

output:
handled
normal flow..

throw

- It is possible for your program to throw an exception explicitly, using the throw statement. The general form of throw is
  - throw ThrowableInstance;

- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.

- Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions.

```
public class Number {
  public static void main(String[] args)
  {
        int a;
        Scanner s=new Scanner(System.in);
        System.out.println("Enter a");
        a=s.nextInt();
        try
        {
           if(a<5)
                throw new Exception("number should be greater than 5");

       }
       catch(Exception e)
       {
          System.out.println(e);
       }

   }

}
```

output:

Enter a

3

java.lang.Exception: number should be greater than 5

Rethrowing an Exception

```
public class Number {
public static void main(String[] args) {
try
        {
try
            {
int a=25/0;
            }
catch(Exception e)
          {
System.out.println("Exception thrown");
throw e;
           }
        }
catch(Exception e)
{
System.out.println("Exception caught");
System.out.println(e);
       }
System.out.println("normal flow..");
   }
 }
```

OUTPUT:

Exception thrown
Exception caught
java.lang.ArithmeticException: / by zero
normal flow..

throws

- If a method is capable of causing an exception that it **does not handle**, it must specify this behavior so that callers of the method can guard themselves against that exception.
- Include a **throws** clause in the method's declaration.
- A throws clause lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause. If they are not, a compile-time error will result.

General form of a method declaration that includes a throws clause:

```
type method-name(parameter-list) throws exception-list
{
// body of method
}
```

Here, exception-list is a comma-separated list of the exceptions that a method can throw.

```
public class Test 1
{
   public static void main(String[] args)
   {
      // TODO code application logic here
      try
      {
         Test t=new Test();
         t.divide();
      }
      catch(IllegalAccessException e)
      {
         System.out.println(e);
      }
   }
}
```

```
Class Test
{
   void divide()throws IllegalAccessException
   {
      throw new IllegalAccessException("Access
           Denied");
   }
   }
```

Output:
java.lang.IllegalAccessException: Access Denied

finally

- it is even possible for an exception to cause the method to return prematurely.
  - For example, if a method opens a file upon entry and closes it upon exit, then you will not want the code that closes the file to be bypassed by the exception-handling mechanism. The finally keyword is designed to address this contingency.
- finally creates a block of code that will be executed after a try/catch block has completed and before the code following the try/catch block.
- **The finally block will execute whether or not an exception is thrown**. If an exception is thrown, the finally block will execute even if no catch statement matches the exception.
- The finally clause is optional. However, each try statement requires at least one catch or a finally clause.
- If a finally block is associated with a try, the finally block will be executed upon conclusion of the try.

Example 1:

```
public class Test
{
    public static void main(String[] args)
   {
      try
      {   int a=10/0;
         System.out.println("NotReachable code");
      }
      catch(ArithmeticException e)
      {
         System.out.println(e);
      }
      catch(ArrayIndexOutOfBoundsException e)
      {          System.out.println(e);
      }
      finally
      {
   System.out.println("This statement will always get executed");
      }
   }
  }
```

Output:
java.lang.ArithmeticException: / by zero
This statement will always get executed

Example 2:

```
public class Test {
public static void main(String[] args) {
      // TODO code application logic here
try
      {
int a=10/0;
System.out.println("NotReachable code");


      }
catch(ArithmeticException e)
      {
System.out.println(e);
      }
finally
      {
System.out.println("This statement will
always get executed");
      }
      Test t=new Test();
t.Method1();
   }
```

```
void Method1()
   {
try
      {
int b[]={1,2,3};
System.out.println("Array Element"+b[5]);


      }
catch(ArrayIndexOutOfBoundsException e)
      {
System.out.println(e);
      }
finally
      {
System.out.println("This statement will always
 get executed in Method1");
      }
   }
 }
```

```
java.lang.ArithmeticException: / by zero
This statement will always get executed
java.lang.ArrayIndexOutOfBoundsException: 5
This statement will always get executed in Method1
```

Creating Your Own Exception Subclasses

- We can create our own exception types to handle situations specific to our applications.
  - define a subclass of Exception (which is, of course, a subclass of Throwable).
- Your subclasses don't need to actually implement anything—it is their existence in the type system that allows you to use them as exceptions.
- The Exception class does not define any methods of its own. It inherit those methods provided by Throwable.
- Thus, all exceptions, including those that you create, have the methods defined by Throwable available to them.

Method	Description
Throwable fillInStackTrace( )	Returns a **Throwable** object that contains a completed stack trace. This object can be rethrown.
Throwable getCause( )	Returns the exception that underlies the current exception. If there is no underlying exception, **null** is returned.
String getLocalizedMessage( )	Returns a localized description of the exception.
String getMessage( )	Returns a description of the exception.
StackTraceElement[ ] getStackTrace( )	Returns an array that contains the stack trace, one element at a time, as an array of **StackTraceElement**. The method at the top of the stack is the last method called before the exception was thrown. This method is found in the first element of the array. The **StackTraceElement** class gives your program access to information about each element in the trace, such as its method name.
Throwable initCause(Throwable *causeExc*)	Associates *causeExc* with the invoking exception as a cause of the invoking exception. Returns a reference to the exception.
void printStackTrace( )	Displays the stack trace.
void printStackTrace(PrintStream *stream*)	Sends the stack trace to the specified stream.
void printStackTrace(PrintWriter *stream*)	Sends the stack trace to the specified stream.
void setStackTrace(StackTraceElement *elements*[ ])	Sets the stack trace to the elements passed in *elements*. This method is for specialized applications, not normal use.
String toString( )	Returns a **String** object containing a description of the exception. This method is called by **println( )** when outputting a **Throwable** object.

TABLE 10-3 The Methods Defined by **Throwable**

- We can override one or more of these methods in exception classes that you create.
- Exception defines four constructors.(2 for chained exceptions)
  - Exception( )
  - Exception(String msg)
- The first form creates an exception that has no description. The second form lets you specify a description of the exception.
- The version of toString( ) defined by Throwable (and inherited by Exception) first displays the name of the exception followed by a colon, which is then followed by your description.
- By overriding toString( ), we can prevent the exception name and colon from being displayed. This makes for a cleaner output, which is desirable in some cases.

```
public class newException extends
Exception{
    int mark;
    newException(int x)
    {
        mark=x;
    }
    public String toString()
    {
        return "newException:" + mark;
    }

}
public class Test {
    public static void main(String[] args)
{
        int m;
        Scanner s=new
Scanner(System.in);
        try
        {
            S.o.p("Enter the mark” );
            m=s.nextInt();
            if(m>=0 && m<=100)
                S.o.p("Mark"+m);
            else
                throw new newException(m);
        }
        catch(newException e)
        {
            System.out.println(e);
        }
        }
}
```

```
Enter the mark
200
newException:200
```

Chained Exceptions

- To allow chained exceptions, two constructors and two methods were added to Throwable.
- The constructors are shown here:
  - Throwable(Throwable causeExc)
  - Throwable(String msg, Throwable causeExc)
- causeExc is the exception that causes the current exception. That is, causeExc is the underlying reason that an exception occurred.
- The second form allows you to specify a description at the same time that you **specify a cause exception.**
- The chained exception methods added to Throwable are getCause( ) and initCause( ).
  - Throwable getCause( )
  - Throwable initCause(Throwable causeExc)
- The getCause( ) method returns the exception that underlies the current exception.
- The initCause( ) method associates causeExc with the invoking exception and returns a reference to the exception.

```
public class Test {
public static void main(String[] args)
{
 try
 {
 NullPointerException n=new NullPointerException("top layer");
  n.initCause(new ArithmeticException("cause"));
   throw n;
   }
      catch(NullPointerException e)
      {
         System.out.println("caught"+e);
          System.out.println("Original Cause"+e.getCause());
      }
    }
}
```

```
caughtjava.lang.NullPointerException: top layer
Original Causejava.lang.ArithmeticException: cause
```

```
public class Matrix
{
   int a[][],b[][],c[][],r1,c1,r2,c2;
      void read()
   {
      Scanner s=new Scanner(System.in);
      System.out.println("Enter the no of rows of first matrix");
       r1=s.nextInt();
        System.out.println("Enter the no of columns of first matrix");
       c1=s.nextInt();
        System.out.println("Enter the no of rows of second matrix");
       r2=s.nextInt();
        System.out.println("Enter the no of columns of second matrix");
       c2=s.nextInt();
 a=new int[r1][c1];
  b=new int[r2][c2];
System.out.println("Enter first matrix");
      for(int i=0;i<r1;i++)
            {
         for(int j=0;j<c1;j++)
         {
            a[i][j]=s.nextInt();
         }
      }
      System.out.println("Enter second matrix");
      for(int i=0;i<r2;i++)
      {
         for(int j=0;j<c2;j++)
         {
            b[i][j]=s.nextInt();
         }
      }
   }
```

```
void multiply()
    {
        int i,j;
            c=new int[r1][c2];
            try
            {
               if(c1!=r2)
               {
                  MyException e=new MyException();
                  throw e;
               }
               else
               {
               for( i = 0; i < r1; i++)
                    {
          for ( j = 0; j < c2; j++)
 {
               for (int k = 0; k < c1; k++)
               {
                  c[i][j] += a[i][k] * b[k][j];
                }
              }
          }
          }
          catch(MyException e)
          {
             System.out.println(e);
          }

      }
      void print()
      {
         System.out.println("Resultant Matrix");
         for(int i=0;i<r1;i++)
         {
            for(int j=0;j<c2;j++)
            {
                System.out.println(c[i][j]);
            }
         }
      }
}
```

```
public class MyException extends Exception
{
   MyException()
   {

   }
   public String toString()
   {
      return "Matrices cannot be multiplied";
   }
   }

public class Test{
   public static void main(String[] args) throws IOException{
    Matrix m=new Matrix();
    m.read();
    m.multiply();
      m.print();
   }
 }
```

Analyzing Stack Trace Elements

- A *stack trace* is a listing of all pending method calls at a particular point in the execution of program.
- You can access the text description of a stack trace by calling the printStackTrace method of the Throwable class.

```
Throwable t = new Throwable();
ByteArrayOutputStream out = new ByteArrayOutputStream();
t.printStackTrace(out);
String description = out.toString();
```

Analyzing Stack Trace Elements

- A more flexible approach is the getStackTrace method that yields an array of StackTraceElement objects, which you can analyze in your program.

  For example:

```
Throwable t = new Throwable();
StackTraceElement[] frames = t.getStackTrace();
for (StackTraceElement frame : frames)
analyze frame
```

- The StackTraceElement class has methods to obtain the file name and line number, as well as the class and method name, of the executing line of code. The toStringmethod yields a formatted string containing all of this information.
- The static Thread.getAllStackTraces method yields the stack traces of all threads.

java.lang.StackTraceElement

- StackTraceElement[] getStackTrace()

  gets the trace of the call stack at the time this object was constructed.

- String getFileName()

  gets the name of the source file containing the execution point of this element, or null if the information is not available.

- int getLineNumber()

  gets the line number of the source file containing the execution point of this element, or -1 if the information is not available.

- String getClassName()

  gets the fully qualified name of the class containing the execution point of this element.

java.lang.StackTraceElement

- String getMethodName()

  gets the name of the method containing the execution point of this element. The name of a constructor is <init>. The name of a static initializer is <clinit>. You can’t distinguish between overloaded methods with the same name.

- boolean isNativeMethod()

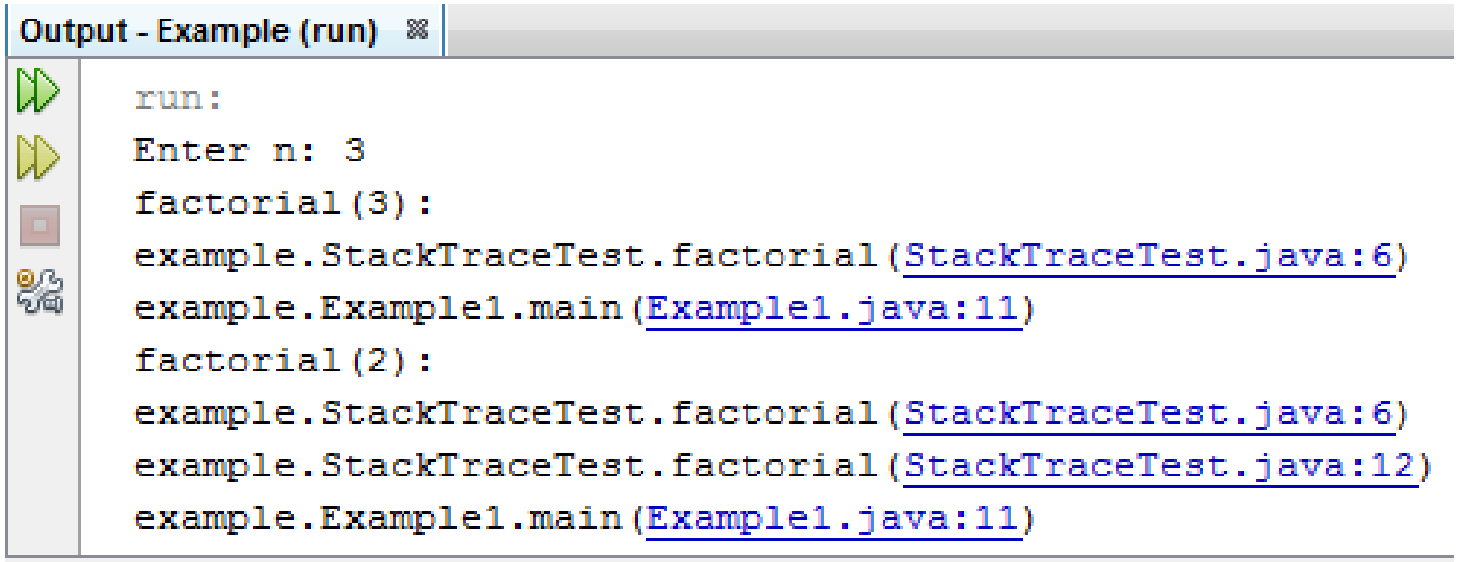
  returns true if the execution point of this element is inside a native method.

- String toString()

  returns a formatted string containing the class and method name and the file name and line number, if available.

```java
package example;
public class StackTraceTest {
public int factorial(int n)
{
System.out.println("factorial(" + n + "):");
Throwable t = new Throwable();
StackTraceElement[] frames = t.getStackTrace();
for (StackTraceElement f : frames)
    System.out.println(f);
int r;
if (n <= 1) r = 1;
else r = n * factorial(n - 1);
System.out.println("return " + r);
return r;
 }
}
```

```java
package example;
import java.util.Scanner;
public class Example1
{
public static void main(String[] args)
    {
  Scanner in = new Scanner(System.in);
System.out.print("Enter n: ");
int n = in.nextInt();
StackTraceTest s=new StackTraceTest();
System.out.println(s.factorial(n));
}
}
```




```
factorial(1):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.Example1.main(Example1.java:11)
return 1
return 2
return 6
6
```

```java
package example;
public class StackTraceTest {
public int factorial(int n)
{
System.out.println("factorial(" + n + "):");
Throwable t = new Throwable();
StackTraceElement[] frames = t.getStackTrace();
for (StackTraceElement f : frames)
    System.out.println(f);
int r;
if (n <= 1) r = 1;
else r = n * factorial(n - 1);
System.out.println("return " + r);
return r;
 }
}
```

```java
package example;
import java.util.Scanner;
public class Example1
{
public static void main(String[] args)
    {
  Scanner in = new Scanner(System.in);
System.out.print("Enter n: ");
int n = in.nextInt();
StackTraceTest s=new StackTraceTest();
System.out.println(s.factorial(n));
}
}
```

Output - Example (run)

```
run:
Enter n: 3
factorial(3):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.Example1.main(Example1.java:11)
factorial(2):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.Example1.main(Example1.java:11)
factorial(1):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.Example1.main(Example1.java:11)
return 1
return 2
return 6
6
```

Output:

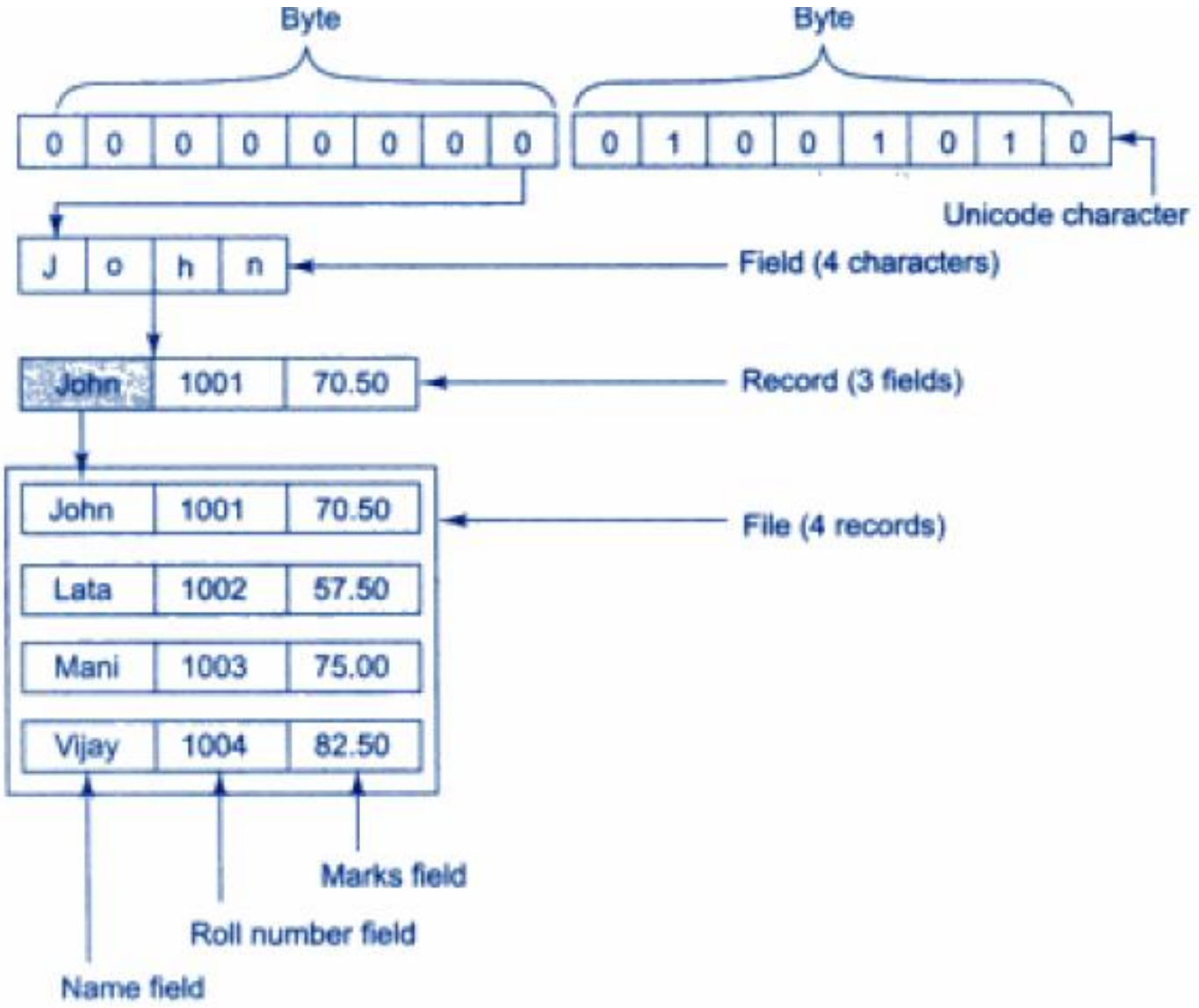
```
Enter n: 3
factorial(3):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.Example1.main(Example1.java:11)
factorial(2):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.Example1.main(Example1.java:11)
factorial(1):
example.StackTraceTest.factorial(StackTraceTest.java:6)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.StackTraceTest.factorial(StackTraceTest.java:12)
example.Example1.main(Example1.java:11)
return 1
return 2
return 6
6
```

Input / Output Basics

Introduction

- Variables and arrays are used for storing data inside the program.
- This storage is temporary.
- Difficult to handle large volume of data using variables and arrays.
- To overcome these by storing data on secondary storage using file.
- Data stored in files is often called persistent data.
- A file is a collection of related records placed in a particular area on the disk.
- A record is composed of several fields and field is a group of characters.
- Characters in Java are Unicode characters composed of two bytes, each byte containing eight binary digits, 1 or 0.

Data representation in Java file



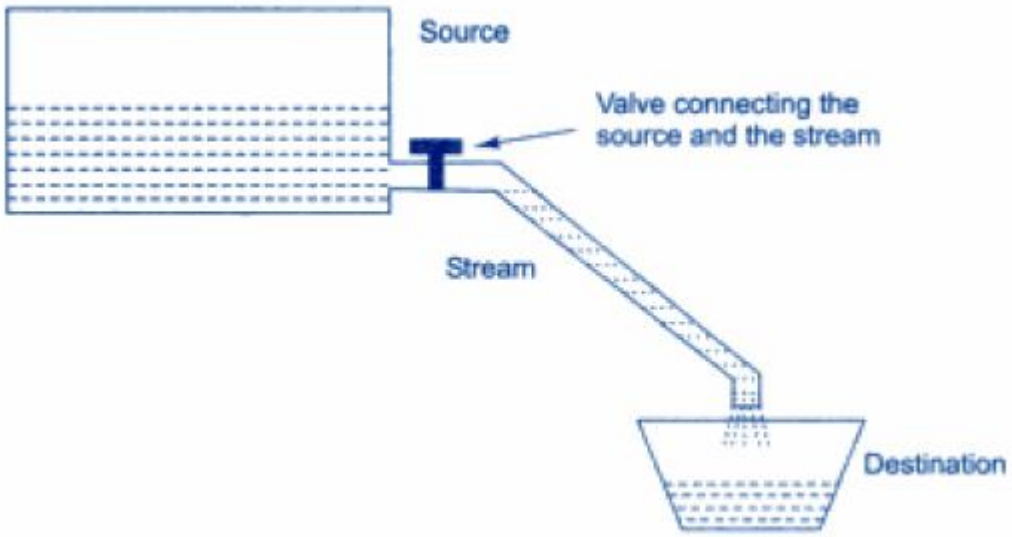
Input / Output Basics

- **Java I/O** (Input and Output) is used *to* ***process the input and produce the output.***
- Java uses the concept of a stream to make I/O operation fast.
- The **java.io** package contains all the classes required for input and output operations.

Stream

- A stream is a sequence of data.

Conceptual view of a stream

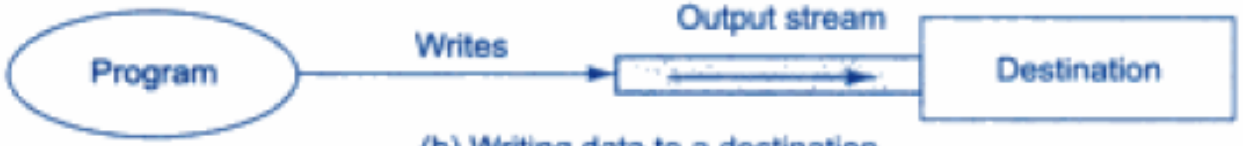


- A stream in Java is a path along which data flows (like a river or a pipe along which water flows).
- It has a source (of data) and a destination (for that data).
- Both the source and the destination may be physical devices or programs or other streams in the same program

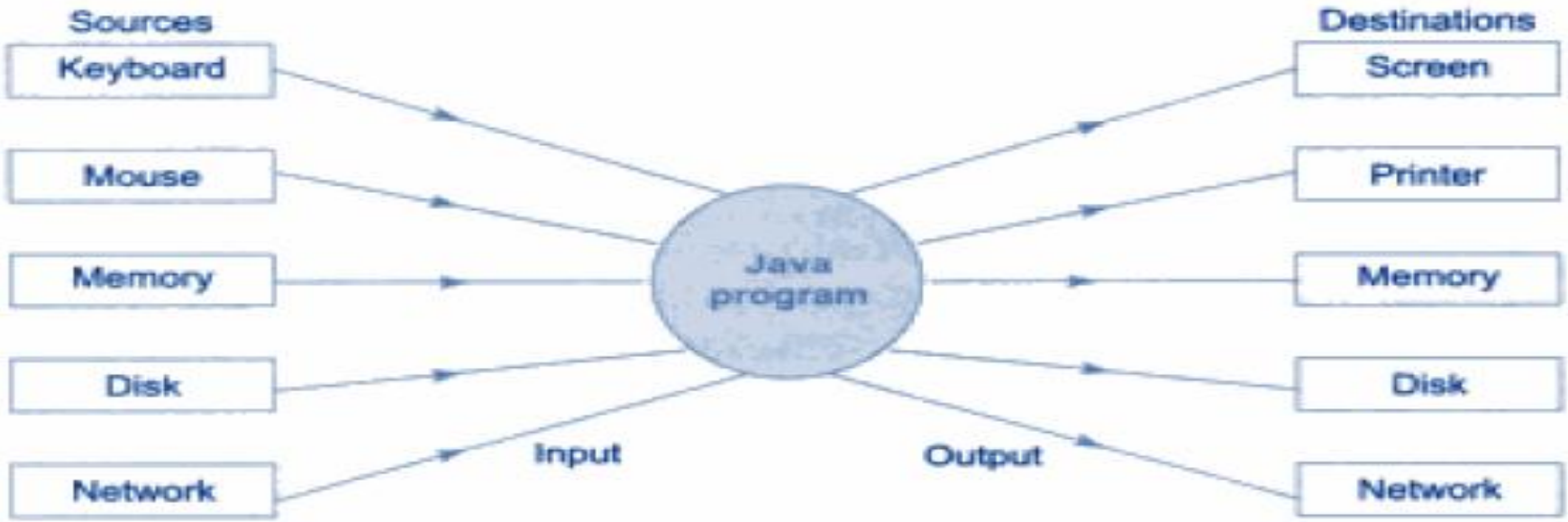
The concept of sending data from one stream to another (like one pipe feeding into another pipe) has made streams in Java a powerful tool for file processing. We can build a complex file processing sequence using a series of simple stream operations. This feature can be used to filter data along the pipeline of streams so that we obtain data in a desired format. For example, we can use one stream to get raw data in binary format and then use another stream in series to convert it to integers.



(a) Reading data into a program



(b) Writing data to a destination

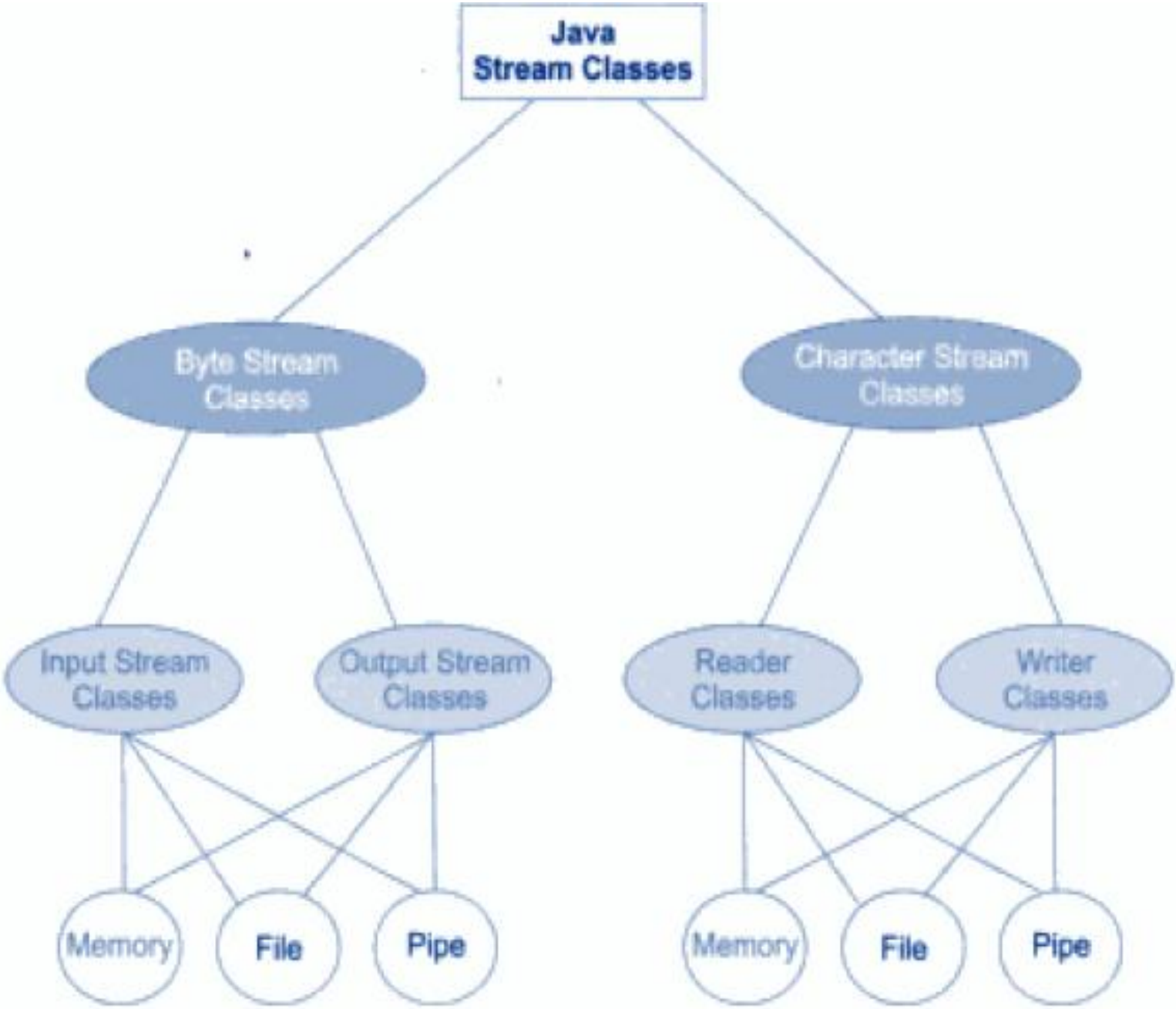


Stream

- In Java, a stream is composed of bytes.
- All Java programs automatically import the java.lang package.
- This package defines a class called **System**, which encapsulates several aspects of the run-time environment.
- For example, using some of its methods, you can obtain the current time and the settings of various properties associated with the system.

Java defines two types of streams: byte and character.

- Byte streams provide a convenient means for handling input and output of bytes.
- Character streams provide a convenient means for handling input and output of characters.


Java
Stream Classes
Byte Stream
Classes
Character Stream
Classes
Input Stream
Classes
Output Stream
Classes
Reader
Classes
Writer
Classes
Memory
File
Pipe
Memory
File
Pipe

Byte Stream Classes

Byte Streams

- **The Byte Stream Classes**
- Byte streams provide a convenient means for handling input and output of bytes.
- Byte streams are used, for example, when reading or writing binary data.
- Java byte streams are used to perform input and output of 8-bit bytes.

Stream

In Java, **3 streams** are created automatically. All these streams are attached with the console.

1) System.out: standard output stream

2) System.in: standard input stream

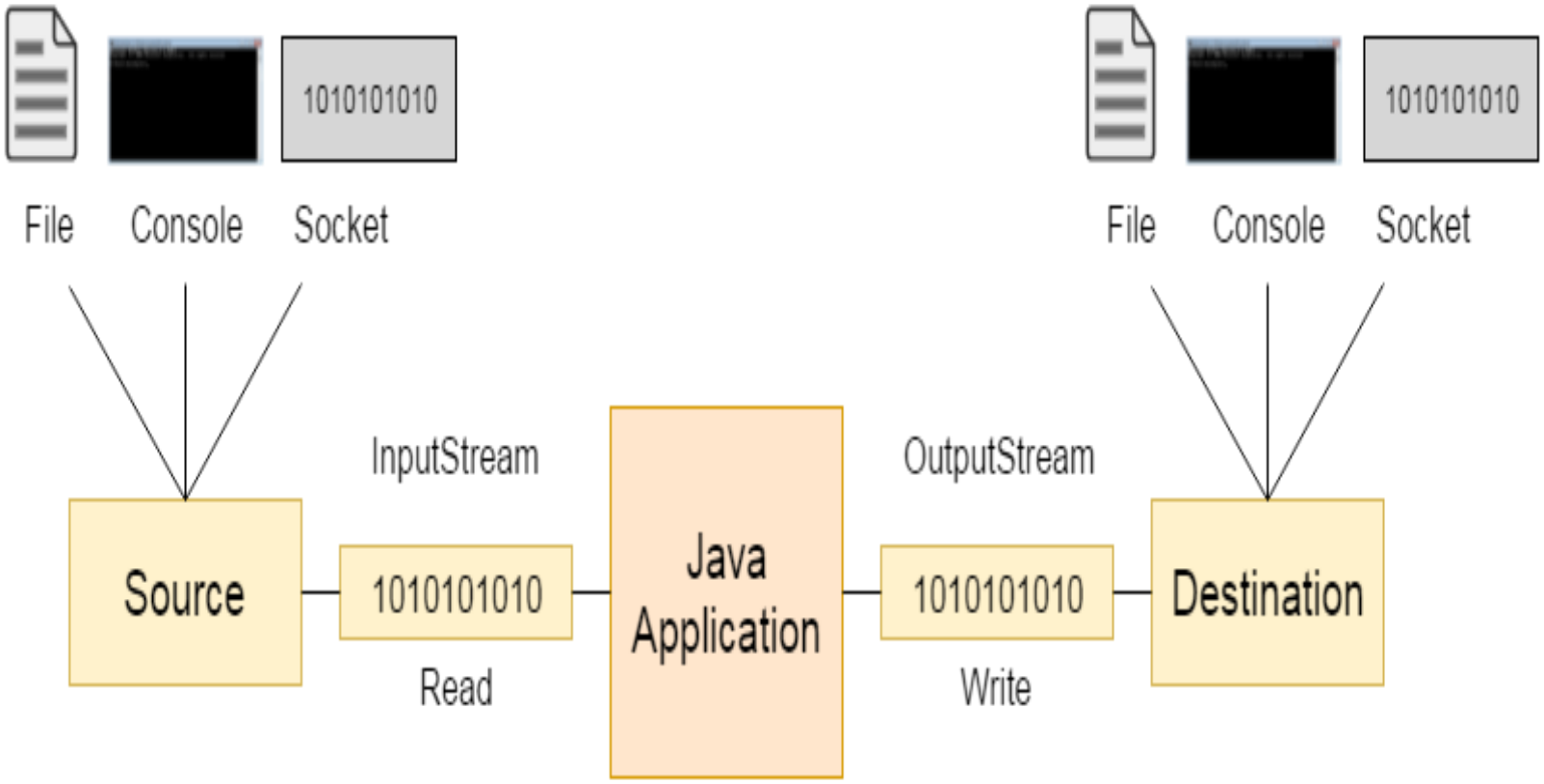
3) System.err: standard error stream

OutputStream

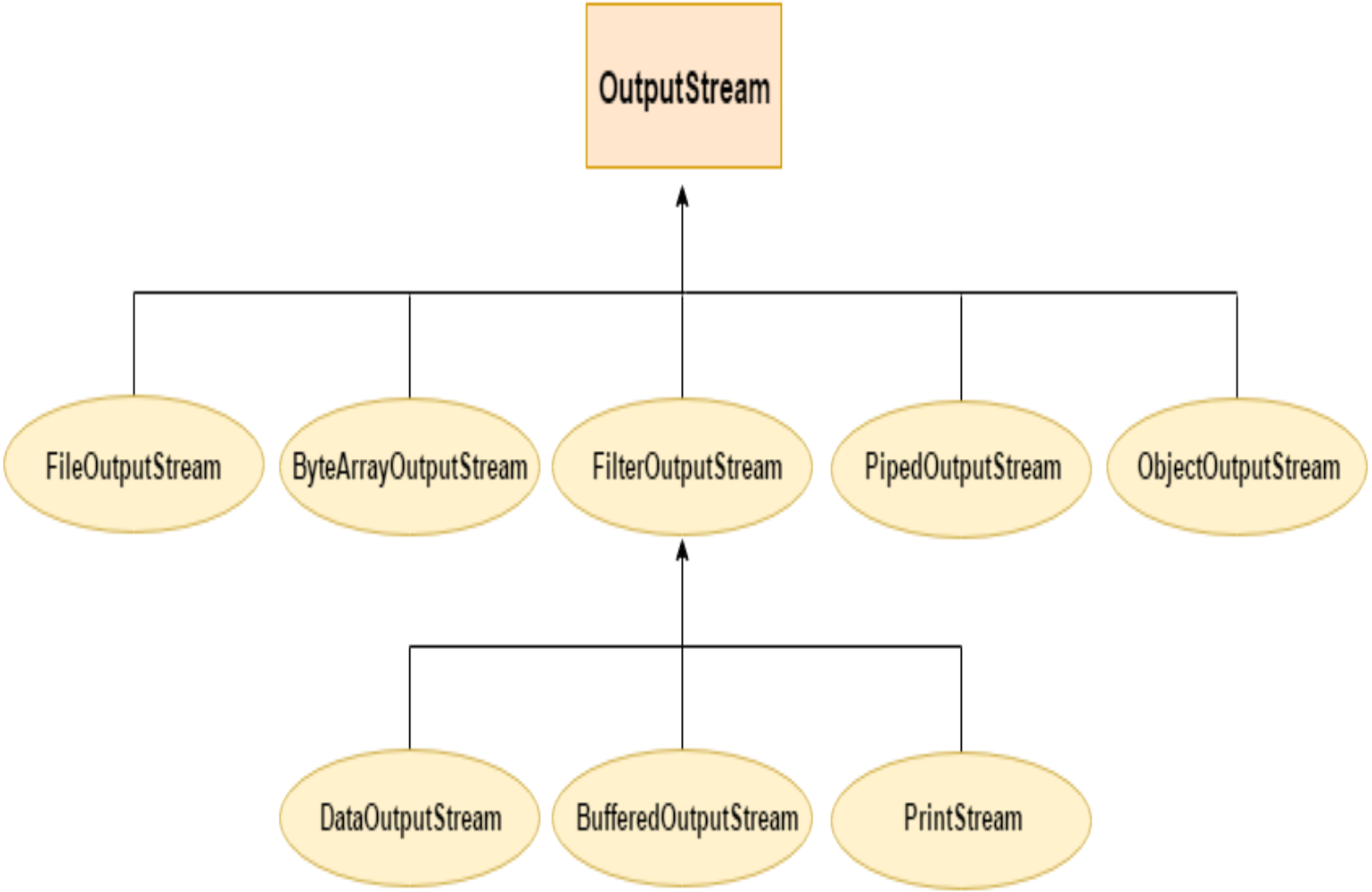
- Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

InputStream

- Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.


1010101010
File
Console
Socket
1010101010
File
Console
Socket
InputStream
OutputStream
Source
1010101010
Read
Java Application
1010101010
Write
Destination

OutputStream Hierarchy

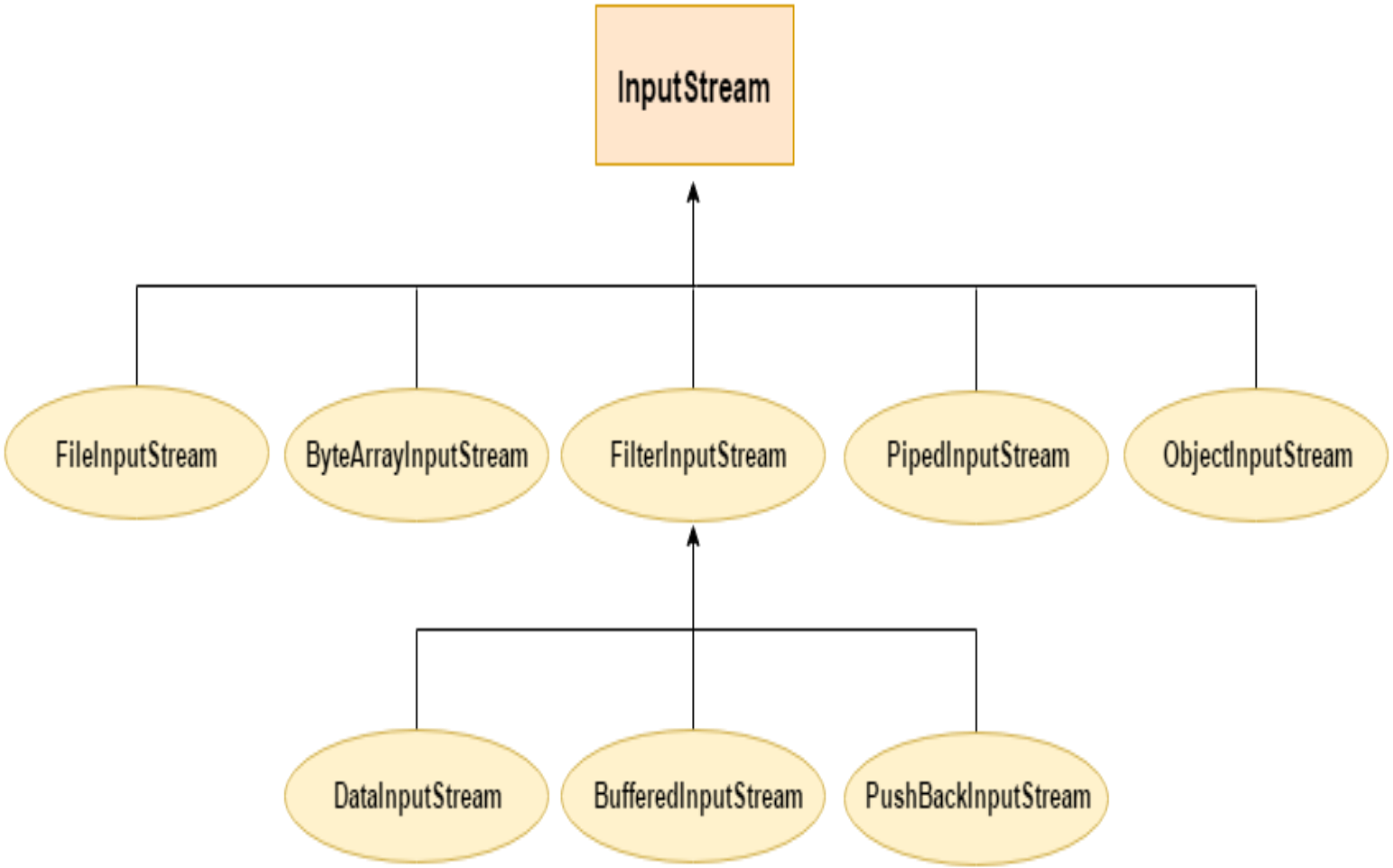


OutputStream class

- OutputStream class is an abstract class.
- It is the superclass of all classes representing an output stream of bytes.
- An output stream accepts output bytes and sends them to some sink.
- **Useful methods of OutputStream**

Method	*Description*
1. write( )	Writes a byte to the output stream
2. write(byte[ ] b)	Writes all bytes in the array b to the output stream
3. write(byte b[ ], int n, int m)	Writes m bytes from array b starting from nth byte
4. close( )	Closes the output stream
5. flush( )	Flushes the output stream

InputStream Hierarchy



InputStream class

- InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.
- **Useful methods of InputStream**

	Method	*Description*
1.	read( )	Reads a byte from the input stream
2.	read (byte b[ ])	Reads an array of bytes into b
3.	read (byte b[ ], int n, int m)	Reads m bytes into b starting from nth byte.
4.	available( )	Gives number of bytes available in the input
5.	skip(n)	Skips over n bytes from the input stream
6.	reset( )	Goes back to the beginning of the stream
7.	close( )	Closes the input stream

Note that the class **DataInputStream** extends **FilterInputStream** and implements the interface **DataInput**. Therefore, the **DataInputStream** class implements the methods described in **DataInput** in addition to using the methods of **InputStream** class. The **DataInput** interface contains the following methods:

- readShort( )
- readInt( )
- readLong( )
- readFloat( )
- readUTF( )
- readDouble( )
- readLine( )
- readChar( )
- readBollean( )

readUTF(): read a string that has been encoded using a modified UTF-8 format

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements **InputStream**
FilterOutputStream	Implements **OutputStream**
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains **print( )** and **println( )**
PushbackInputStream	Input stream that supports one-byte "unget," which returns a byte to the input stream
RandomAccessFile	Supports random access file I/O
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

Character Stream Classes

The Character Stream Classes

- Character streams can be used to read and write 16-bit Unicode characters
- Character streams are defined by using two class hierarchies.
- At the top are two abstract classes, Reader and Writer.
- These abstract classes handle Unicode character streams.
- Java has several concrete subclasses of each of these.

Reader & Writer Stream Classes

- Reader stream classes are designed to read characters from files.
- These classes are functionally very similar to the Input Stream classes, except Input Streams uses bytes as their fundamental unit of information, while reader streams uses characters.
- Like output stream classes, the writer stream classes are designed to perform all output operations on files.
- Only difference is that, write stream classes are designed to write characters.

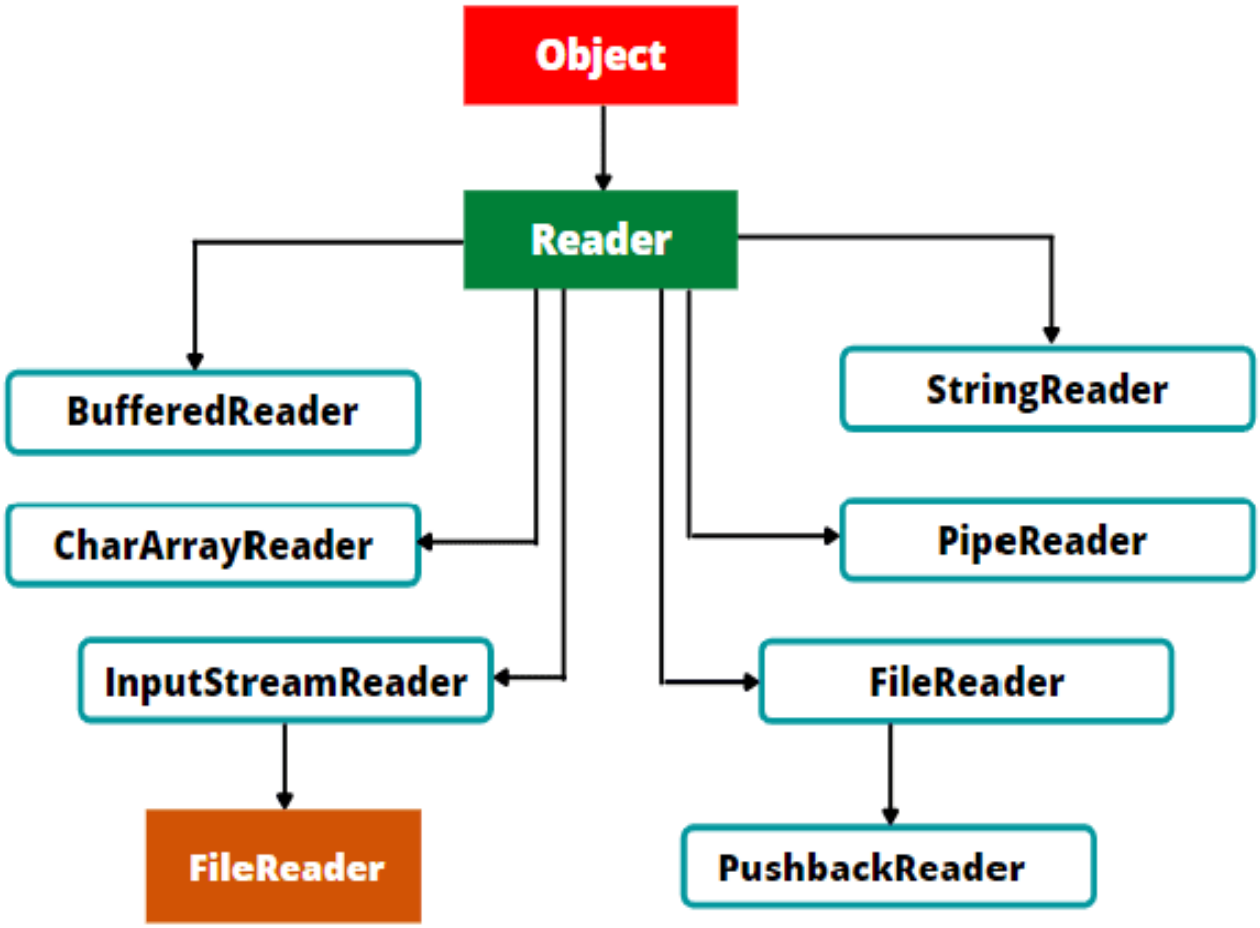


Fig: Hierarchy of Reader Stream Classes

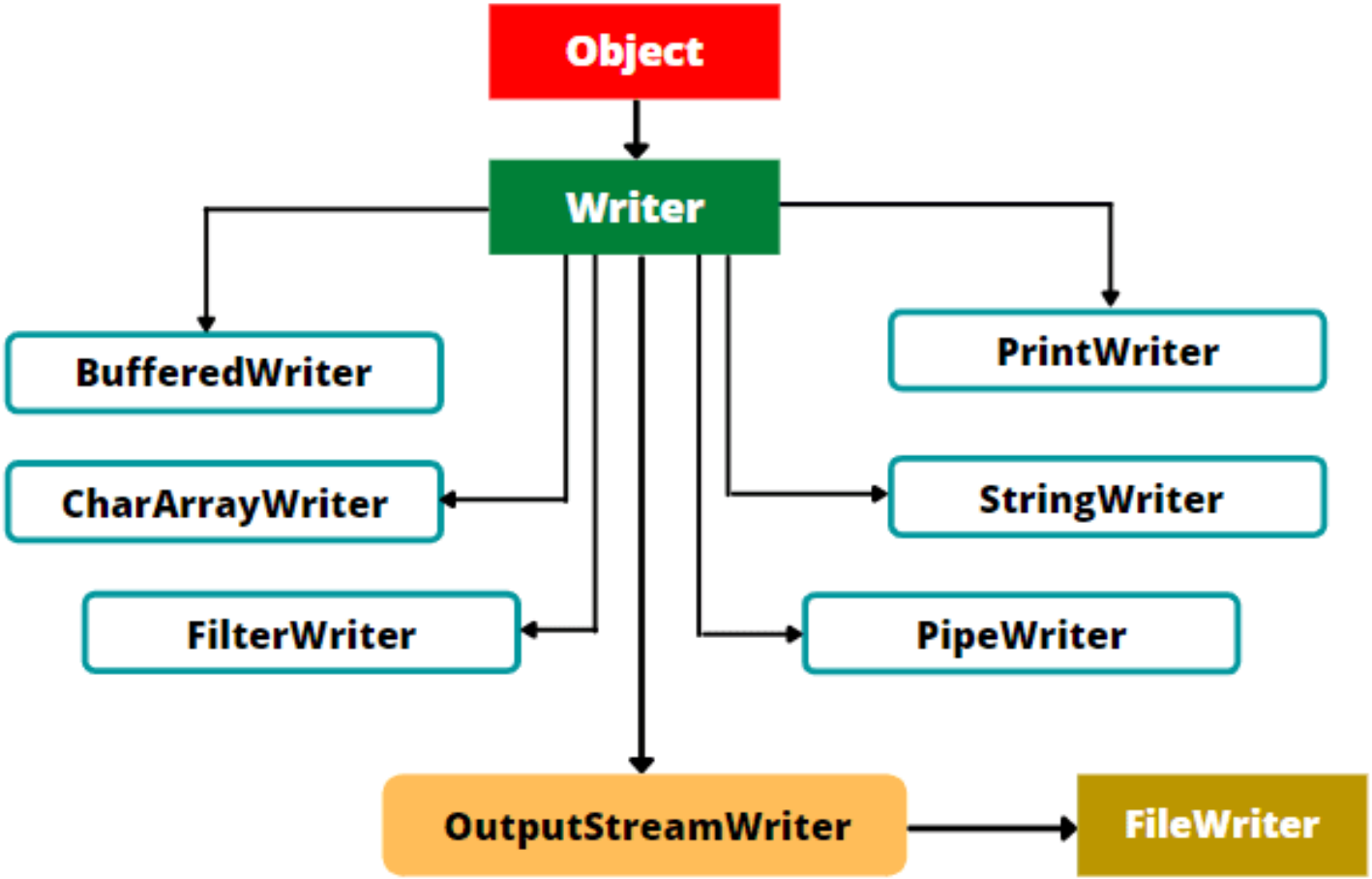


Fig: Hierarchy of Writer Stream Classes

The Character Stream Classes

- The character stream classes are shown in Table below.

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer

Stream Class	Meaning
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains **print( )** and **println( )**
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

Note: To use the stream classes, you must import java.io.

Reading Console Input

Reading Console Input

In java there are four ways to read input from the console.

They are

1.using BufferedReader

2.using DataInputStream

3.using Scanner

4.using Console

BufferedReader class

- In Java, console input is accomplished by reading from System.in. To obtain a character based stream that is attached to the console, wrap System.in in a BufferedReader object.
- BufferedReader supports a buffered input stream. Its most commonly used constructor is shown here:

BufferedReader(Reader inputReader)

- Here, inputReader is the stream that is linked to the instance of BufferedReader that is being created. Reader is an abstract class. One of its concrete subclasses is InputStreamReader, which converts bytes to characters. To obtain an InputStreamReader object that is linked to System.in, use the following constructor:

InputStreamReader(InputStream inputStream)

- Because System.in refers to an object of type InputStream, it can be used for inputStream.
- Putting it all together, the following line of code creates a BufferedReader that is connected to the keyboard:

```
BufferedReader br = new BufferedReader(new
                    InputStreamReader(System.in));
```

- After this statement executes, br is a character-based stream that is linked to the console through System.in.

BufferedReader class

Reading Characters

- To read a character from a BufferedReader, use read( ).

General form of read() is :

int read( ) throws IOException

- Each time that read( ) is called, it reads a character from the input stream and returns it as an integer value.
- It returns –1 when the end of the stream is encountered.
- As you can see, it can throw an IOException.

BufferedReader class

- The following program demonstrates read( ) by reading characters from the console until the user types a "q.” Notice that any I/O exceptions that might be generated are simply thrown out of main( ).

```
import java.io.*;
public class Example
{
public static void main(String[] args)throws IOException
{
char c;
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter characters, 'q' to quit.");
do
 {
c = (char) br.read();
System.out.println(c);
} while(c != 'q');
  }}
```

Output:

```
Enter characters, 'q' to quit.
abcdq
a
b
c
d
q
```

BufferedReader class

Reading Strings

- To read a string from the keyboard, use the version of readLine( ) that is a member of the BufferedReader class.
- Its general form is shown here:

String readLine( ) throws IOException

- It returns a String object

BufferedReader class

Reading Strings

Example1:

```
import java.io.*;
public class Example
{
public static void main(String[] args)throws IOException
{
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String str;
System.out.println("Enter lines of text.");
System.out.println("Enter 'stop' to quit.");
do
{
str = br.readLine();
System.out.println(str);
} while(!str.equals("stop"));
}
}
```

Output:

```
Enter lines of text.
Enter 'stop' to quit.
india is my country
india is my country
stop
stop
```

Example 2- creating a tiny text editor.

- It creates an array of String objects and then reads in lines of text, storing each line in the array. It will read up to 100 lines or until you enter “stop.”

```
import java.io.*;
public class Example
 {
public static void main(String[] args)throws IOException
{
BufferedReaderbr = new BufferedReader(new InputStreamReader(System.in));
String str[] = new String[100];
System.out.println("Enter lines of text.");
System.out.println("Enter 'stop' to quit.");
for(int i=0; i<100; i++)
 {
str[i] = br.readLine();
if(str[i].equals("stop")) break;
}
System.out.println("\nHere is your file:");
// display the lines
for(int i=0; i<100; i++)
 {
if(str[i].equals("stop")) break;
System.out.println(str[i]);
}
}}
```

Output:

```
Enter lines of text.
Enter 'stop' to quit.
India is my country
All Indians are my brothers and
sisters
stop

Here is your file:
India is my country
All Indians are my brothers and
sisters
```

DataInputStream

- Java DataInputStream class allows an application to read primitive data from the input stream in a machine-independent way.
- Java application generally uses the data output stream to write data that can later be read by a data input stream.
- **Java DataInputStream class Methods**

Method	Description
int read(byte[] b)	It is used to read the number of bytes from the input stream.
int read(byte[] b, int off, int len)	It is used to read len bytes of data from the input stream.
int readInt()	It is used to read input bytes and return an int value.
byte readByte()	It is used to read and return the one input byte.
char readChar()	It is used to read two input bytes and returns a char value.
double readDouble()	It is used to read eight input bytes and returns a double value.
boolean readBoolean()	It is used to read one input byte and return true if byte is non zero, false if byte is zero.

DataInputStream

```
public class Test
{
public static void main(String[] args) throws IOException
{
        DataInputStream dis = new DataInputStream(System.in);
        System.out.println("Enter a number");
        int i=Integer.parseInt(dis.readLine());
        System.out.println(i);
        System.out.println("Enter a string");
         String s=dis.readLine();
        System.out.println(s);
  }
 }
```

Output:

```
Enter a number
2
2
Enter a string
aaa
aaa
```

Scanner class

- The Scanner class is a class in java.util, which allows the user to read values of various types.

Class Constructors:

- There are two constructors that are particularly useful: one takes an InputStream object as a parameter and the other takes a FileReader object as a parameter.

```
Scanner in = new Scanner(System.in);  // System.in is an InputStream
Scanner inFile = new Scanner(new FileReader("myFile"));
```

Scanner class: Methods

Method	Returns
int nextInt()	Returns the next token as an int. If the next token is not an integer, InputMismatchException is thrown.
long nextLong()	Returns the next token as a long. If the next token is not an integer, InputMismatchException is thrown.
float nextFloat()	Returns the next token as a float. If the next token is not a float or is out of range, InputMismatchException is thrown.
double nextDouble()	Returns the next token as a long. If the next token is not a float or is out of range, InputMismatchException is thrown.
String next()	Finds and returns the next complete token from this scanner and returns it as a string; a token is usually ended by whitespace such as a blank or line break. If not token exists, NoSuchElementException is thrown.
String nextLine()	Returns the rest of the current line, excluding any line separator at the end.
void close()	Closes the scanner.
boolean hasNextLine()	Returns true if the scanner has another line in its input; false otherwise.
boolean hasNextInt()	Returns true if the next token in the scanner can be interpreted as an int value.
boolean hasNextFloat()	Returns true if the next toke in the scanner can be interpreted as a float value.

Program:

```
public class Test
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
char gender = sc.next().charAt(0);
int age = sc.nextInt();
long mobileNo = sc.nextLong();
double cgpa = sc.nextDouble();
System.out.println("Name: "+name);
System.out.println("Gender: "+gender);
System.out.println("Age: "+age);
System.out.println("Mobile Number: "+mobileNo);
System.out.println("CGPA: "+cgpa);
    }
  }
```

Java Console class

- The Java Console class is be used to get input from console.
- It provides methods to read texts and passwords.
- If you read password using Console class, it will not be displayed to the user.
- The java.io.Console class is attached with system console internally.
- Console supplies no constructors. Instead, a Console object is obtained by calling System.console( )

Java Console class declaration

- Let's see the declaration for Java.io.Console class:

public final class Console extends Object implements Flushable

Java Console class methods

Input Methods:

- readLine(): Reads a single line of text from the console.
- readLine(String fmt, Object... args): Displays a formatted prompt and then reads a single line of text from the console.
- readPassword(): Reads a password or sensitive data from the console without echoing the input characters.
- readPassword(String fmt, Object... args): Displays a formatted prompt and then reads a password or sensitive data from the console without echoing the input characters.

Output Methods:

- printf(String fmt, Object... args): Writes a formatted string to the console's output stream using the specified format string and arguments, similar to C's printf.
- format(String fmt, Object... args): A convenience method that functions identically to printf().

Java Console class methods

- **Stream Access Methods:**
- reader(): Retrieves the unique Reader object associated with this console, allowing for more advanced character-based input.
- writer(): Retrieves the unique PrintWriter object associated with this console, allowing for more advanced character-based output.
- **Utility Methods:**
- flush(): Flushes the console and forces any buffered output to be written immediately.
- charset(): Returns the Charset object used for the console.

Program:

```
public class  Test
{
            public static void main(String[] args) throws IOException{
 String n;
System.out.println("Enter your name: ");
             n=System.console().readLine();
            System.out.println("Welcome "+n);
   }
}
```

output:

```
Enter your name:
java
Welcome java
```

Program:

```
import java.io.*;
class ConsoleDemo
{
   public static void main(String args[])
   {
      String str;
            Console con = System.console();
if(con == null)
      {
         System.out.print("No console available");
         return;
      }
   str = con.readLine("Enter your name: ");
      con.printf("Here is your name: %s\n", str);
System.out.println("Enter the password: ");
      char[] ch=con.readPassword();
      //converting char array into string
      String pass = String.valueOf(ch);
      System.out.println("Password is: " + pass);
   }
}
```

Writing Console Output

- Console output is most easily accomplished with print( ) and println( ).
- These methods are defined by the class PrintStream (which is the type of object referenced by System.out).
- Even though System.out is a byte stream, using it for simple program output is still acceptable.
- Because PrintStream is an output stream derived from OutputStream, it also implements the low-level method write( ). Thus, write( ) can be used to write to the console. The simplest form of write( ) defined by PrintStream is shown here:

  void write(int byteval)

- This method writes to the stream the byte specified by byteval. Although byteval is declared as an integer, only the low-order eight bits are written.

Example:

```
public class Example
{
public static void main(String[] args)
{
int b;
b = 'A';
System.out.write(b);
System.out.write('\n');
}
}
```

Output:

```
A
```

The PrintWriter Class

- For real-world programs, the recommended method of writing to the console when using Java is through a PrintWriter stream.
- PrintWriter is one of the character-based classes. Using a character-based class for console output makes it easier to internationalize your program.
- PrintWriter defines several constructors.

PrintWriter(OutputStream outputStream, boolean flushOnNewline)

- Here, outputStream is an object of type OutputStream, and flushOnNewline controls whether Java flushes the output stream every time a println( ) method is called.
- If flushOnNewline is true, flushing automatically takes place.
- If false, flushing is not automatic.

The PrintWriter Class

- PrintWriter supports the print( ) and println( ) methods for all types including Object.
- Thus, you can use these methods in the same way as they have been used with System.out.
- If an argument is not a simple type, the PrintWriter methods call the object's toString( ) method and then print the result.
- To write to the console by using a PrintWriter, specify System.out for the output stream and flush the stream after each newline.
- For example, this line of code creates a PrintWriter that is connected to console output:

```
PrintWriter pw = new PrintWriter(System.out, true);
```

Example:

```
import java.io.*;
public class Example
{
public static void main(String[] args)
{
PrintWriter pw = new PrintWriter(System.out, true);
pw.println("This is a string");
int i = -7;
pw.println(i);
double d = 4.5;
pw.println(d);
}
}
```

Output:

```
This is a string
-7
4.5
```

Reading and Writing Files

Reading and Writing Files

- Java provides a number of classes and methods that allow you to read and write files.
- In Java, all files are byte-oriented, and Java provides methods to read and write bytes from and to a file.
- However, Java allows you to wrap a byte-oriented file stream within a character-based object.
- Two of the most often-used stream classes are FileInputStream and FileOutputStream, which create byte streams linked to files.
- To open a file, you simply create an object of one of these classes, specifying the name of the file as an argument to the constructor. While both classes support additional, overridden constructors, the following are the forms that we will be using:
- FileInputStream(String fileName) throws FileNotFoundException
- FileOutputStream(String fileName) throws FileNotFoundException

Reading and Writing Files

- FileInputStream(String fileName) throws FileNotFoundException
- FileOutputStream(String fileName) throws FileNotFoundException
- When you create an input stream, if the file does not exist, then FileNotFoundException is thrown.
- For output streams, if the file cannot be created, then FileNotFoundException is thrown.
- When an output file is opened, any preexisting file by the same name is destroyed.
- When you are done with a file, you should close it by calling close( ).
- It is defined by both FileInputStream and FileOutputStream, as shown here:
- void close( ) throws IOException

Reading from a file

- To read from a file, you can use a version of read( ) that is defined within FileInputStream.
- The one that we will use is shown here:

  int read( ) throws IOException

- Each time that it is called, it reads a single byte from the file and returns the byte as an integer value.
- read( ) returns –1 when the end of the file is encountered.
- It can throw an IOException.

```
import java.io.*;
public class Example
{
public static void main(String[] args)throws IOException
   {
int i;
FileInputStream fin;
try {
fin = new FileInputStream("J:/AA.txt");
   }
catch(FileNotFoundException e)
   {
System.out.println("File Not Found");
return;
   } // read characters until EOF is encountered
do {
   i = fin.read();
if(i != -1) System.out.print((char) i);
   } while(i != -1);
fin.close(); }}
```

Output:
Hello How are you?

Writing to a file

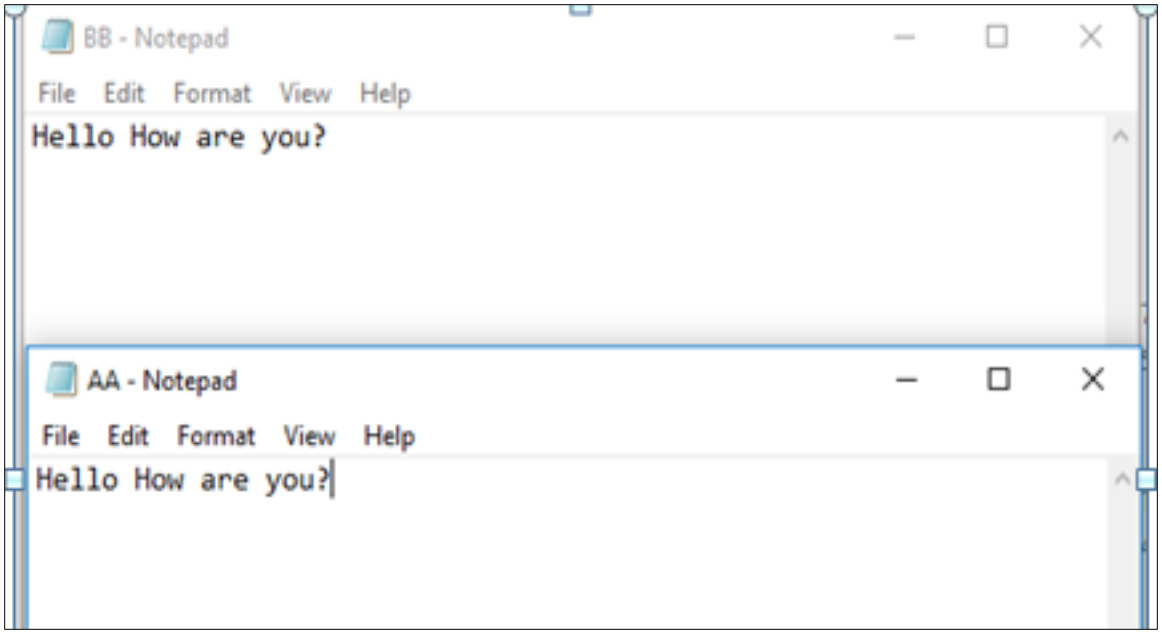
- To write to a file, you can use the write( ) method defined by FileOutputStream.
- Its simplest form is shown here:

  void write(intbyteval) throws IOException
- This method writes the byte specified by byteval to the file.
- Although byteval is declared as an integer, only the low-order eight bits are written to the file.
- If an error occurs during writing, an IOException is thrown.

```
import java.io.*;
public class Example
{
public static void main(String[] args)throws IOException
   {
int i;
FileInputStream fin;
FileOutputStreamfout;
try
 {
fin = new FileInputStream("J:/AA.txt");
}
catch(FileNotFoundException e)
{
System.out.println("Input File Not Found");
return;
}
```

```
// open output file
try
{
fout = new FileOutputStream("J:/BB.txt");
}
catch(FileNotFoundException e)
 {
System.out.println("Error Opening Output File");
return;
}
// Copy File
try
{
do
{
i = fin.read();
if(i != -1) fout.write(i);
} while(i != -1);
}
```

```
catch(IOException e)
 {
System.out.println("File Error");
}
fin.close();
fout.close();
}
}
```



```
// Java program to count the
// number of lines, words, sentences,
// characters, and whitespaces in a file
import java.io.*;

public class Test {
    public static void main(String[] args)
        throws IOException
    {
        File file = new File("C:\\Users\\hp\\Desktop\\TextReader.txt");
        FileInputStream fileInputStream = new FileInputStream(file);
        InputStreamReader inputStreamReader = new
InputStreamReader(fileInputStream);
        BufferedReader bufferedReader = new BufferedReader(inputStreamReader);
        String line;
        int wordCount = 0;
        int characterCount = 0;
        int paraCount = 0;
        int whiteSpaceCount = 0;
        int sentenceCount = 0;
```

```
while ((line = bufferedReader.readLine()) != null) {
        if (line.equals("")) {
            paraCount += 1;
        }
        else {
            characterCount += line.length();
            String words[] = line.split("\\s+");
            wordCount += words.length;
            whiteSpaceCount += wordCount - 1;
            String sentence[] = line.split("[!?.:]+");
            sentenceCount += sentence.length;
        }
    }
    if (sentenceCount >= 1) {
        paraCount++;
    }
    System.out.println("Total word count = "+ wordCount);
    System.out.println("Total number of sentences = "+ sentenceCount);
    System.out.println("Total number of characters = "+ characterCount);
    System.out.println("Number of paragraphs = "+ paraCount);
    System.out.println("Total number of whitespaces = "+ whiteSpaceCount);
  }
}
```